



Département Mathématiques, Informatique et Réseaux

Agent MCP CRM Bitext

Groupe :

- Supervisé par :

- Juin 2026

Table des matières

Introduction	3
1 Contexte, objectifs et données	4
1.1 Sélection et analyse des datasets	4
1.1.1 Telco Customer Churn	4
1.1.2 92k Call Center Scripts	5
1.1.3 Bitext Customer Support Chatbot	5
2 Architecture et processus de réalisation	6
2.1 Dossier data	6
2.2 Dossier src	6
2.3 Dossier scripts	7
2.4 Étapes du processus de réalisation	7
2.4.1 Étape 1 : téléchargement ou placement du dataset	7
2.4.2 Étape 2 : préparation des données	8
2.4.3 Étape 3 : analyse exploratoire	8
2.5 Architecture MCP	9
3 Modèles de classification et génération	11
3.1 Classificateur principal : TF-IDF + LogisticRegression	11
3.2 Justification du choix	11
3.3 Variante Transformer	11
3.4 Choix du modèle Ollama	12
4 Agent, API et interface utilisateur	13
4.1 API FastAPI	13
4.1.1 Endpoint /health	14
4.1.2 Endpoint /chat	14
4.1.3 Endpoint /intents	14
4.1.4 Gestion des sessions	14
4.2 Interface Streamlit	15
4.2.1 Fonctionnalités principales	16
4.2.2 Seuil de confiance	16
4.2.3 Dashboard de session	16
4.2.4 Exemple de test	16
5 Dépendances et déploiement	20
5.1 requirements.txt	20
5.2 requirements_light.txt	20

5.3	Déploiement	20
5.3.1	Lancement local	20
5.3.2	Mode sans Ollama	20
5.3.3	Mode avec Ollama	21
5.3.4	Docker	21
6	Évaluation, bilan et perspectives	23
6.1	Évaluation du classificateur	23
6.2	Tests fonctionnels	23
6.3	Tests de cas limites	23
6.4	État final d'avancement	23
6.4.1	Éléments finalisés	23
6.4.2	Éléments partiellement réalisés	24
6.5	Limites actuelles	24
6.6	Perspectives d'amélioration	24
6.7	Liste des captures d'écran à intégrer	25
	Conclusion générale	26
	Résumé final	26

Introduction

Ce rapport présente la finalisation du projet académique mini-projet **INEAgent MCP CRM Bitext**, un système conversationnel intelligent appliqué au domaine du support client et de la relation client. Le projet vise à concevoir un agent capable de comprendre un message utilisateur, d'identifier son intention, de router cette intention vers le bon outil métier, puis de produire une réponse adaptée. Il a été réalisé par les étudiants :

- BENABOU Hafsa,
- COMPAORE Moustapha,
- et DIALLO Mamadou Tahirou

Tous étudiants en 2eme année de cycle ingénieur Data à INPT.

L'objectif initial était de développer une solution agentique basée sur le **Model Context Protocol** (MCP), en combinant :

- un dataset CRM spécialisé ;
- un classificateur d'intentions ;
- des tools métier simulant des actions CRM ;
- une API FastAPI ;
- une interface utilisateur Streamlit ;
- un modèle local Ollama pour reformuler ou générer des réponses naturelles.

Le projet a évolué progressivement depuis une première phase de conception et d'expérimentation dans un notebook, jusqu'à une version structurée sous forme de projet Python modulaire, avec scripts, API, interface graphique et documentation.

Chapitre 1

Contexte, objectifs et données

Le contexte du projet est celui de l'automatisation intelligente du support client. Dans un service CRM, les utilisateurs peuvent exprimer plusieurs types de demandes :

- annuler une commande ;
- suivre une livraison ;
- demander une facture ;
- changer une adresse de livraison ;
- demander un remboursement ;
- contacter un agent humain ;
- vérifier les moyens de paiement ;
- déposer une réclamation.

L'objectif du projet est donc de mettre en place un agent capable de traiter ce type de messages de manière structurée.

Message utilisateur -> Classification de l'intention -> Selection du tool MCP correspondant -> Execution du tool CRM -> Generation ou reformulation de la reponse finale

Cette architecture permet de séparer clairement la compréhension du message, la décision métier, l'exécution d'une action CRM et la génération de réponse. Cette séparation rend le système plus modulaire, plus testable et plus facilement améliorable.

1.1 Sélection et analyse des datasets

Au début du projet, plusieurs datasets ont été analysés afin de choisir la source de données la plus adaptée à un agent CRM.

1.1.1 Telco Customer Churn

Le dataset **Telco Customer Churn** contient environ 7 043 clients et 21 variables structurées. Il est pertinent pour des tâches de prédiction du churn, c'est-à-dire l'identification des clients susceptibles de quitter un service.

Son rôle potentiel dans le projet était d'alimenter un futur tool MCP de scoring client :

```
predict_churn(customer_id)
```

Ce dataset est utile pour enrichir le contexte CRM, mais il ne répond pas directement au besoin principal de classification d'intentions conversationnelles.

1.1.2 92k Call Center Scripts

Le dataset **92k Call Center Scripts** contient environ 91 706 transcripts réels de centres d'appels. Il offre une richesse conversationnelle importante et pourrait être utilisé pour construire une mémoire conversationnelle ou un système RAG.

Son rôle potentiel était de servir de corpus documentaire pour enrichir les réponses de l'agent. Cependant, sa structure est moins directement exploitable pour un pipeline simple de classification d'intentions.

1.1.3 Bitext Customer Support Chatbot

Le dataset **Bitext Customer Support Chatbot** contient environ 26 900 paires instruction/réponse, réparties en 27 intentions et 11 catégories.

Il a été retenu comme dataset principal parce qu'il est directement aligné avec le besoin du projet :

- classification d'intentions ;
- entraînement supervisé ;
- routage vers des actions CRM ;
- génération de réponses orientées support client.

Le dataset Bitext est donc devenu le pivot du projet.

Chapitre 2

Architecture et processus de réalisation

La solution finale repose sur une architecture modulaire organisée autour des composants suivants :

```
Dataset Bitext
-> Preparation et nettoyage des donnees
-> Classificateur d'intentions
-> Mapping intent -> tool MCP
-> Agent CRM
-> API FastAPI
-> Interface Streamlit
```

Le projet est structuré autour de plusieurs fichiers et dossiers :

```
data/
src/
scripts/
api.py
streamlit_app.py
requirements.txt
requirements_light.txt
Dockerfile
README.md
```

2.1 Dossier data

Le dossier `data/` contient le dataset source, les fichiers `train/validation/test`, les fichiers JSONL, le classificateur entraîné et éventuellement un modèle Transformer sauvegardé.

2.2 Dossier src

Le dossier `src/` contient le coeur logique du projet :

- `data.py` : chargement, nettoyage et préparation des données ;
- `classifier.py` : classificateur TF-IDF + LogisticRegression ;
- `transformer_classifier.py` : wrapper pour un modèle Transformer ;
- `mcp_mapping.py` : mapping des intentions vers les tools MCP ;

- `tools.py` : stubs fonctionnels des tools CRM ;
- `agent.py` : orchestration de l'agent CRM ;
- `ollama.py` : fonctions de connexion à Ollama ;
- `config.py` : configuration globale du projet.

2.3 Dossier scripts

Le dossier `scripts/` contient les scripts d'exécution :

- `download_dataset.py` ;
- `prepare_data.py` ;
- `train_classifier.py` ;
- `train_transformer_classifier.py` ;
- `evaluate_classifier.py` ;
- `demo_agent.py` ;
- `check_ollama.py`.

2.4 Étapes du processus de réalisation

2.4.1 Étape 1 : téléchargement ou placement du dataset

Une contribution importante a été l'ajout du script :

```
scripts/download_dataset.py
```

Ce script permet de télécharger automatiquement le dataset Bitext depuis Hugging Face :

```
load_dataset("bitext/Bitext-customer-support-llm-chatbot-training-dataset")
```

Le fichier est ensuite sauvegardé dans :

```
data/Bitext_Sample_Customer_Support_Training_Dataset_27K_responses-v11_1.csv
```

Cette étape améliore la reproductibilité du projet, car l'utilisateur n'a plus besoin de placer manuellement le fichier CSV dans le dossier `data/`.

Capture 1 – Téléchargement du dataset

Exécution de `python scripts/download_dataset.py`.

Emplacement réservé pour capture d'écran

FIGURE 2.1 – Capture 1 – Téléchargement du dataset

2.4.2 Étape 2 : préparation des données

La préparation des données est réalisée avec :

```
scripts/prepare_data.py
```

Cette étape effectue plusieurs traitements :

- chargement du CSV Bitext ;
- remplacement des templates comme {{Order Number}} ou {{Website URL}} ;
- création de colonnes nettoyées ;
- split stratifié train/validation/test ;
- export des fichiers CSV ;
- génération des fichiers JSONL.

Les fichiers générés sont :

```
data/bitext_train.csv  
data/bitext_val.csv  
data/bitext_test.csv  
data/bitext_train_formatted.jsonl  
data/bitext_val_formatted.jsonl
```

Capture 2 – Préparation des données

Exécution de `python scripts/prepare_data.py` et génération des fichiers.

Emplacement réservé pour capture d'écran

FIGURE 2.2 – Capture 2 – Préparation des données

2.4.3 Étape 3 : analyse exploratoire

L'analyse exploratoire a permis d'étudier la distribution des intentions, les catégories, les flags comportementaux, les longueurs de textes et les templates présents dans les messages.

Capture 3 – Distribution des catégories et des intentions

Graphique d'analyse exploratoire du dataset.

Emplacement réservé pour capture d'écran

FIGURE 2.3 – Capture 3 – Distribution des catégories et des intentions

Capture 4 – Distribution des longueurs de texte

Graphique des longueurs des instructions et réponses.

Emplacement réservé pour capture d'écran

FIGURE 2.4 – Capture 4 – Distribution des longueurs de texte

Capture 5 – Matrice de confusion

Fichier `confusion_matrix.png`.

Emplacement réservé pour capture d'écran

FIGURE 2.5 – Capture 5 – Matrice de confusion

2.5 Architecture MCP

Le projet repose sur un mapping explicite entre les intentions du dataset Bitext et des tools MCP. Ce mapping est défini dans :

```
src/mcp_mapping.py
```

Exemples :

```
cancel_order -> cancel_order(order_id)
track_order -> get_order_status(order_id)
change_shipping_address -> update_shipping_address(order_id, new_address)
get_refund -> process_refund(order_id, reason)
contact_human_agent -> escalate_to_human(reason)
```

Les tools sont implémentés dans :

```
src/tools.py
```

Ils sont actuellement simulés, mais fonctionnels. Ils retournent des dictionnaires Python représentant des réponses CRM.

Capture 6 – Mapping intent vers tool MCP

Extrait du mapping ou exemple d'appel d'un tool.

Emplacement réservé pour capture d'écran

FIGURE 2.6 – Capture 6 – Mapping intent vers tool MCP

Chapitre 3

Modèles de classification et génération

3.1 Classificateur principal : TF-IDF + LogisticRegression

Le classificateur principal du projet repose sur :

```
TF-IDF + LogisticRegression
```

Il est implémenté dans :

```
src/classifier.py
```

Le pipeline utilise des unigrammes et bigrammes, un maximum de 10 000 features, une régression logistique et un entraînement supervisé sur la colonne **intent**.

Commande d'entraînement :

```
python scripts/train_classifier.py
```

Le modèle entraîné est sauvegardé dans :

```
data/intent_classifier.pkl
```

Ce choix est particulièrement adapté aux contraintes matérielles du projet, car il fonctionne efficacement sur CPU.

3.2 Justification du choix

Au départ, le projet envisageait un fine-tuning de modèle LLM, notamment avec QLoRA sur Qwen/Qwen2.5-3B-Instruct. Cependant, cette approche demande un GPU, comme une carte T4 sur Google Colab.

Or, l'environnement local disponible est principalement CPU. Le choix final a donc été d'utiliser **TF-IDF + LogisticRegression** pour la classification d'intentions, et de réserver Ollama à la reformulation de réponse.

3.3 Variante Transformer

Le projet intègre également une variante Transformer optionnelle. Les fichiers concernés sont :

```
src/transformer_classifier.py
scripts/train_transformer_classifier.py
```

Le modèle par défaut est :

```
google/bert_uncased_L-2_H-128_A-2
```

Il s'agit d'un mini-BERT léger, plus adapté au CPU qu'un grand modèle Transformer. Le script permet également d'utiliser DistilBERT :

```
python scripts/train_transformer_classifier.py --model-name distilbert-base-uncased
```

Cette variante est intéressante pour expérimenter avec des modèles deep learning, mais elle reste plus lourde que TF-IDF.

3.4 Choix du modèle Ollama

Le modèle Ollama recommandé dans la version finale est :

```
qwen2.5:1.5b
```

Ce choix a été fait car il représente un bon compromis entre qualité de génération et performance sur CPU.

La configuration se trouve dans :

```
src/config.py
```

avec :

```
DEFAULT_OLLAMA_MODEL = "qwen2.5:1.5b"
FALLBACK_OLLAMA_MODEL = "qwen2.5:1.5b"
```

Le rôle d'Ollama dans la solution finale n'est pas de classifier l'intention. La classification est assurée par TF-IDF ou Transformer. Ollama sert uniquement à produire ou reformuler une réponse plus naturelle.

Capture 7 – Vérification Ollama

Vérification du modèle `qwen2.5:1.5b` installé localement.

Emplacement réservé pour capture d'écran

FIGURE 3.1 – Capture 7 – Vérification Ollama

Chapitre 4

Agent, API et interface utilisateur

La classe principale de l'agent est :

```
src/agent.py
```

Elle est représentée par :

```
CRMAgent
```

Son rôle est d'orchestrer tout le pipeline :

1. recevoir le message utilisateur ;
2. classifier l'intention ;
3. vérifier le score de confiance ;
4. appliquer un fallback vers un agent humain si la confiance est faible ;
5. exécuter le tool MCP correspondant ;
6. générer ou retourner une réponse ;
7. stocker l'historique de conversation.

Capture 8 – Test de l'agent

Test de l'agent en ligne de commande avec `scripts/demo_agent.py`.

Emplacement réservé pour capture d'écran

FIGURE 4.1 – Capture 8 – Test de l'agent

4.1 API FastAPI

L'API est implémentée dans :

```
api.py
```

Elle expose plusieurs endpoints :

```
GET /health
POST /chat
DELETE /session/{session_id}
GET /intents
```

4.1.1 Endpoint /health

Cet endpoint permet de vérifier que l'API est active.

4.1.2 Endpoint /chat

Cet endpoint permet d'envoyer un message à l'agent CRM.

```
curl -X POST http://localhost:8000/chat ^
-H "Content-Type: application/json" ^
-d "{\"message\":\"I want to cancel my order number 12345\",\"session_id\":\"demo\"}"
```

4.1.3 Endpoint /intents

Cet endpoint permet de consulter la liste des intentions connues par le classificateur.

4.1.4 Gestion des sessions

L'API maintient des sessions en mémoire avec un dictionnaire Python. Cela permet de garder un historique conversationnel par session.

Capture 9 – Lancement de l'API

Terminal avec `uvicorn api:app -reload -port 8000`.

Emplacement réservé pour capture d'écran

FIGURE 4.2 – Capture 9 – Lancement de l'API

Capture 10 – Swagger FastAPI

Interface /docs.

Emplacement réservé pour capture d'écran

FIGURE 4.3 – Capture 10 – Swagger FastAPI

Capture 11 – Test du endpoint /chat

Requête de test d'un message client.

Emplacement réservé pour capture d'écran

FIGURE 4.4 – Capture 11 – Test du endpoint /chat

Capture 12 – Liste des intents

Résultat du endpoint /intents.

Emplacement réservé pour capture d'écran

FIGURE 4.5 – Capture 12 – Liste des intents

4.2 Interface Streamlit

Une contribution importante de finalisation a été l'ajout d'une interface utilisateur interactive avec Streamlit. Le fichier concerné est :

```
streamlit_app.py
```

Cette interface permet de rendre le système plus accessible, plus démontrable et plus compréhensible. Elle est connectée à l'API FastAPI et permet de tester le pipeline complet sans passer par la ligne de commande.

4.2.1 Fonctionnalités principales

L'interface permet :

- de configurer l'URL de l'API FastAPI ;
- de vérifier si l'API est connectée ;
- de consulter la liste des intents disponibles ;
- de saisir un message client personnalisé ;
- de sélectionner un exemple prédéfini ;
- d'envoyer le message à l'API ;
- d'afficher l'intention détectée ;
- d'afficher le score de confiance ;
- d'afficher le tool MCP appelé ;
- de consulter le résultat technique du tool ;
- de suivre l'historique des interactions ;
- d'exporter l'historique au format CSV.

4.2.2 Seuil de confiance

L'interface intègre un seuil de confiance configurable, fixé par défaut à 0.70. Si le score de confiance est inférieur à ce seuil, l'interface recommande une redirection vers un agent humain.

4.2.3 Dashboard de session

Un dashboard permet de suivre le nombre de messages analysés, la confiance moyenne, le nombre d'intentions détectées, le nombre de cas nécessitant une escalade, la répartition des intentions et l'historique technique.

4.2.4 Exemple de test

Un exemple de test concluant a été réalisé avec le message :

I want to cancel my order number 12345

Le système a détecté l'intention `cancel_order` et a appelé le tool `cancel_order`.

Capture 13 – Interface Streamlit

Page d'accueil de l'interface.

Emplacement réservé pour capture d'écran

FIGURE 4.6 – Capture 13 – Interface Streamlit

Capture 14 – Configuration API

Sidebar avec URL API, session ID et état de connexion.

Emplacement réservé pour capture d'écran

FIGURE 4.7 – Capture 14 – Configuration API

Capture 15 – Message client

Saisie d'un message client.

Emplacement réservé pour capture d'écran

FIGURE 4.8 – Capture 15 – Message client

Capture 16 – Résultat de classification

Intent, score de confiance et niveau.

Emplacement réservé pour capture d'écran

FIGURE 4.9 – Capture 16 – Résultat de classification

Capture 17 – Tool MCP appelé

Affichage du tool appelé et du résultat technique.

Emplacement réservé pour capture d'écran

FIGURE 4.10 – Capture 17 – Tool MCP appelé

Capture 18 – Dashboard Streamlit

Statistiques de session.

Emplacement réservé pour capture d'écran

FIGURE 4.11 – Capture 18 – Dashboard Streamlit

Capture 19 – Export CSV

Bouton de téléchargement de l'historique.

Emplacement réservé pour capture d'écran

FIGURE 4.12 – Capture 19 – Export CSV

Chapitre 5

Dépendances et déploiement

Deux fichiers de dépendances sont disponibles.

5.1 requirements.txt

Ce fichier contient les dépendances complètes, notamment `pandas`, `matplotlib`, `seaborn`, `scikit-learn`, `fastapi`, `uvicorn`, `pydantic`, `langchain`, `langchain-ollama`, `datasets`, `transformers`, `torch` et `evaluate`.

5.2 requirements_light.txt

Ce fichier contient une version plus légère pour l'API et l'interface Streamlit. Il est plus adapté aux machines à ressources limitées lorsqu'on ne souhaite pas entraîner de modèle Transformer.

5.3 Déploiement

5.3.1 Lancement local

Pour lancer l'API :

```
uvicorn api:app --reload --port 8000
```

Pour lancer Streamlit :

```
streamlit run streamlit_app.py
```

5.3.2 Mode sans Ollama

```
$env:CRM_AGENT_USE_OLLAMA="false"  
uvicorn api:app --reload --port 8000
```

5.3.3 Mode avec Ollama

```
ollama serve  
ollama pull qwen2.5:1.5b  
uvicorn api:app --reload --port 8000
```

5.3.4 Docker

```
docker build -t agent-crm .  
docker run -p 8000:8000 -e CRM_AGENT_USE_OLLAMA=false agent-crm
```

Capture 20 – Lancement FastAPI

Terminal de lancement de FastAPI.

Emplacement réservé pour capture d'écran

FIGURE 5.1 – Capture 20 – Lancement FastAPI

Capture 21 – Lancement Streamlit

Terminal de lancement de Streamlit.

Emplacement réservé pour capture d'écran

FIGURE 5.2 – Capture 21 – Lancement Streamlit

Capture 22 – Build Docker

Commande `docker build`.

Emplacement réservé pour capture d'écran

FIGURE 5.3 – Capture 22 – Build Docker

Capture 23 – Conteneur Docker

Commande `docker run`.

Emplacement réservé pour capture d'écran

FIGURE 5.4 – Capture 23 – Conteneur Docker

Chapitre 6

Évaluation, bilan et perspectives

6.1 Évaluation du classificateur

Le script :

```
scripts/evaluate_classifier.py
```

permet de calculer l'accuracy globale, la confiance moyenne, le ratio d'exemples avec confiance supérieure à 80%, un rapport de classification et une matrice de confusion.

6.2 Tests fonctionnels

Les tests fonctionnels portent sur la préparation des données, l'entraînement du classificateur, le chargement du modèle, l'appel des tools MCP, le fonctionnement de l'API, le fonctionnement de l'interface Streamlit, le routage correct des intents et la gestion des faibles scores de confiance.

6.3 Tests de cas limites

Les cas limites prévus incluent :

- message vide ;
- message très court ;
- faute de frappe ;
- intention multiple ;
- demande hors périmètre ;
- message agressif ;
- absence de numéro de commande.

6.4 État final d'avancement

6.4.1 Éléments finalisés

- Le notebook initial a été transformé en projet Python modulaire.

- Le dataset Bitext est intégré dans un workflow reproductible.
- Le script de téléchargement du dataset a été ajouté.
- Les données peuvent être préparées automatiquement.
- Le classificateur TF-IDF est entraînable localement.
- Le modèle est sauvegardé dans `data/intent_classifier.pkl`.
- L'agent CRM est implémenté.
- Le mapping intent vers tool MCP est complet.
- Les tools MCP sont simulés et fonctionnels.
- L'API FastAPI est connectée au vrai agent.
- L'interface Streamlit est disponible.
- Ollama est configuré avec `qwen2.5:1.5b`.
- Le mode sans Ollama est disponible.
- Le mode Transformer optionnel est disponible.
- La documentation README a été structurée en étapes.

6.4.2 Éléments partiellement réalisés

- Le fine-tuning complet d'un LLM avec QLoRA n'a pas été retenu pour l'exécution locale CPU.
- Le dataset Telco Churn n'a pas encore été intégré comme tool MCP.
- Les tools CRM restent des stubs, non connectés à un vrai système métier.
- Le modèle Transformer reste optionnel et expérimental.
- La persistance des sessions est en mémoire, sans Redis ou base de données.

6.5 Limites actuelles

Le projet reste perfectible sur plusieurs points :

- la classification peut être moins robuste sur des messages réels très ambigus ;
- les intentions multiples ne sont pas encore gérées ;
- l'extraction des paramètres repose sur des règles simples ;
- les tools MCP ne sont pas connectés à un vrai CRM ;
- la mémoire conversationnelle est limitée ;
- l'interface Streamlit pourrait encore exposer plus d'options de configuration.

6.6 Perspectives d'amélioration

Plusieurs pistes sont possibles pour faire évoluer le projet :

- intégrer le dataset Telco Churn ;
- ajouter un tool `predict_churn(customer_id)` ;
- améliorer l'extraction d'entités ;

- gérer les intentions multiples ;
- connecter les stubs MCP à un vrai backend CRM ;
- ajouter une persistance Redis ou SQLite ;
- améliorer l'interface Streamlit ;
- calibrer les seuils de confiance ;
- tester DistilBERT ou une architecture embeddings.

6.7 Liste des captures d'écran à intégrer

Capture	Description
1	Téléchargement du dataset avec <code>download_dataset.py</code> .
2	Préparation des données avec <code>prepare_data.py</code> .
3	Fichiers générés dans le dossier <code>data/</code> .
4	Distribution des catégories et intentions.
5	Matrice de confusion.
6	Entraînement du classificateur.
7	Vérification Ollama.
8	Démo agent en ligne de commande.
9	Lancement FastAPI.
10	Swagger FastAPI.
11	Test du endpoint <code>/chat</code> .
12	Interface Streamlit : accueil.
13	Configuration API dans Streamlit.
14	Test d'un message client.
15	Résultat de classification.
16	Tool MCP appelé.
17	Dashboard Streamlit.
18	Historique technique.
19	Export CSV.
20	Build Docker.
21	Conteneur Docker lancé.

Conclusion générale

Le projet **Agent MCP CRM Bitext** a évolué d'un notebook expérimental vers une application modulaire complète. La solution finale permet de traiter des messages clients, de détecter leur intention, d'appeler un tool CRM correspondant, puis de produire une réponse exploitable.

La solution repose sur une architecture pragmatique adaptée aux ressources disponibles :

```
TF-IDF + LogisticRegression pour la classification
Tools MCP pour la logique metier
FastAPI pour l'exposition du service
Streamlit pour la demonstration interactive
Ollama qwen2.5:1.5b pour la reformulation optionnelle
```

Le projet atteint donc son objectif principal : démontrer un agent CRM intelligent, modulaire et fonctionnel, capable de combiner classification d'intentions, routage MCP et interaction utilisateur.

Les contributions finales ont permis de renforcer la reproductibilité, l'ergonomie et la démontrabilité du projet, notamment grâce au script de téléchargement automatique du dataset, à l'intégration d'un backend Transformer optionnel, au choix d'un modèle Ollama plus léger, à l'interface Streamlit, à la structuration complète du projet et à la documentation par étapes.

Même si certains points restent perfectibles, notamment l'intégration d'un vrai CRM et la gestion des intentions multiples, la version actuelle constitue une base solide pour une solution agentique CRM locale et extensible.

Résumé final

Le projet finalisé peut être résumé ainsi :

```
Un agent CRM intelligent capable de :
- comprendre une demande client ;
- identifier son intention ;
- appeler un outil metier MCP ;
- retourner une reponse structuree ;
- etre teste via API ou interface Streamlit ;
- fonctionner sur CPU avec un modele léger.
```

La solution est adaptée aux contraintes matérielles du projet et reste évolutive pour de futures améliorations.